Project - Submission Sheet

Software Testing - Random Testing

Names: Kavya
Group: Group 4

Software Testing - Summer 2024 - May 9, 2025 - Sibylle Schupp / Daniel Rashedi

Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

Answer:

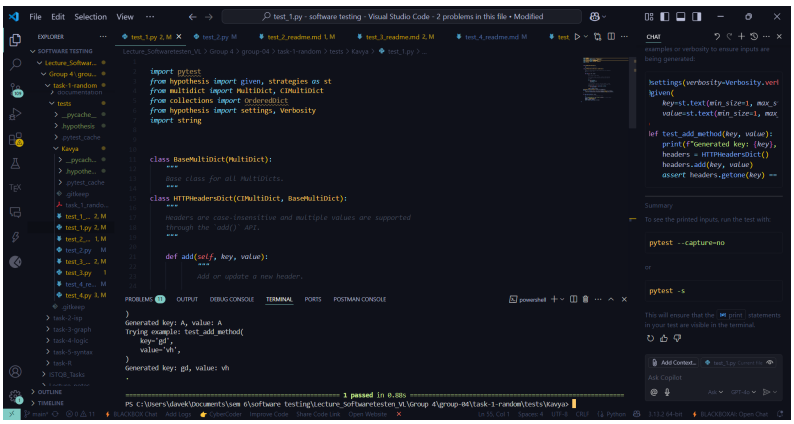

Figure 1: Execution of test_add_method

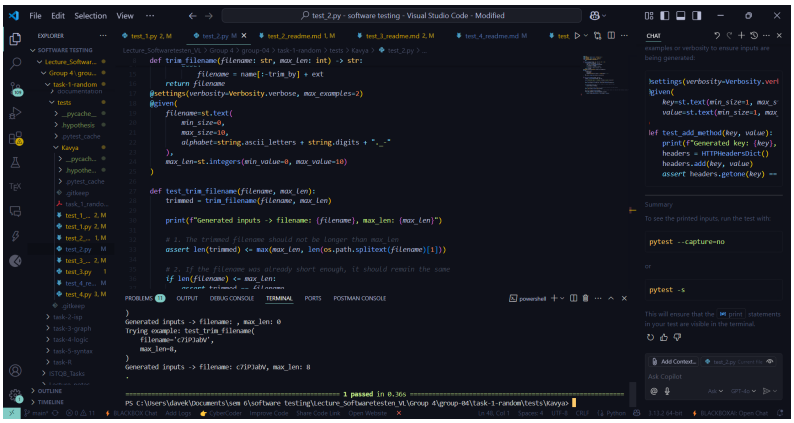

Figure 2: Execution of test_trim_filename

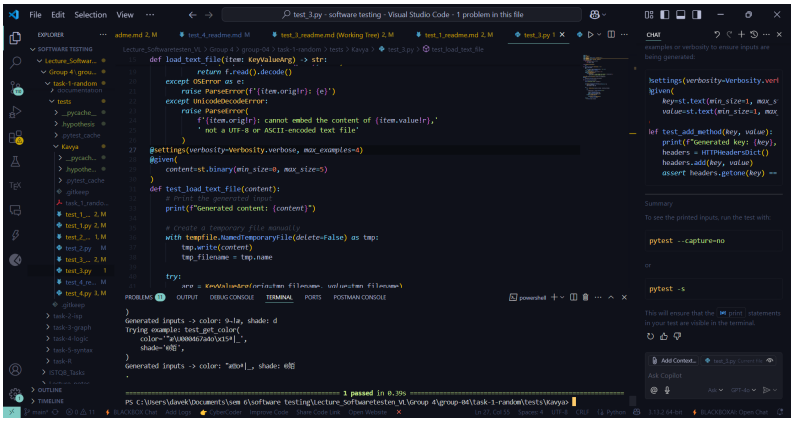

Figure 3: Execution of test_load_text_file

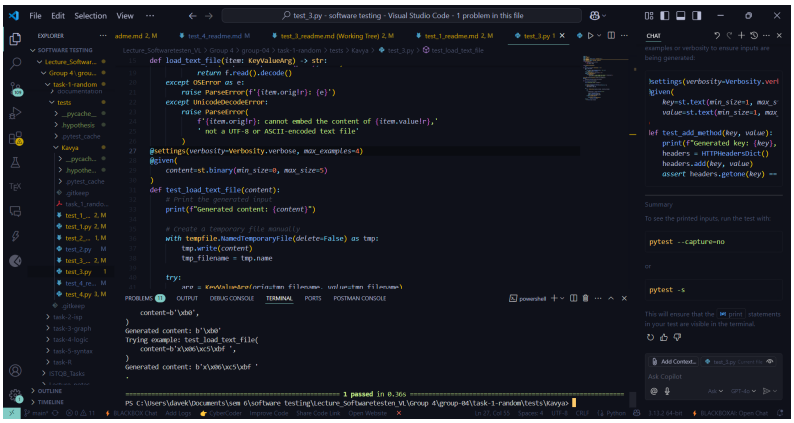

Figure 4: Execution of test_get_color

c) Assess the input values generated by `hypothesis`. Do the tests represent meaningful scenarios?:

Answer:

Test	Input Strategy	Realistic	Edge Cases Cove
`test_add_method`	`st.text(min_size=1, max_size=10)`	Yes	Yes
`test_trim_filename`	`st.text(...), st.integers(0, 300)`	Yes	Yes
`test_load_text_file`	`st.binary(min_size=0, max_size=100)`	Yes	Yes
`test_get_color`	`st.text(min_size=1, max_size=10)`	Partially	Yes

Table 1: Assessment of Hypothesis-Generated Test Inputs Across 4 Functions

...

AI Review Task

Assess the correctness and understandability of the LLM's generated tests applying the paper's methodology. Add any code relevant for your assessment to the answer:

Answer:

Group: 4 LM Studio

Prompt 1:

```
import os
from hypothesis import given, strategies as st
import string
from hypothesis import settings, Verbosity
class ContentRangeError(ValueError):
pass

def trim_filename(filename: str, max_len: int) -> str:
if len(filename) > max_len:
trim_by = len(filename) - max_len
name, ext = os.path.splitext(filename)
if trim_by >= len(name):
based on this generate 4 test cases
```

Prompt 2: :Assess the aspects of correctness and understandability of your above test cases in comparison to the LLM's generated test cases according to the paper's methodology described in its RQ1

Group: Your Assessment

$Basedontheprovidedcodeandrequirements, herearefourtestcasesforthetrim_{f}ilenamefunction:$

Test Case 1 - Exact Match

```
@given(filename="test.txt", max_len=8)
def test_trim_filename(filename, max_len):
    # Test that the filename remains unchanged if its length matches max_len
    assert trim_filename(filename, max_len) == "test.txt"
```

Test Case 2 - Partial Extension Preservation

```
@given(filename="longfile.dox", max_len=5)
def test_trim_filename(filename, max_len):
# Test that at least part of the extension is preserved if possible
trimmed = trim_filename(filename, max_len)
assert len(trimmed) == 6  # "long.dox"
```

Test Case 3 - No Extension

```
@given(filename="shortname", max_len=7)
def test_trim_filename(filename, max_len):
# Test that filenames without extensions are handled correctly
trimmed = trim_filename(filename, max_len)
assert trimmed == "shortname"
```

Test Case 4 - Full Trimming

```
@given(filename="VeryLongName", max_len=0)
def test_trim_filename(filename, max_len):
# Test that the filename is not modified when max_len
    is 0
trimmed = trim_filename(filename, max_len)
assert trimmed == "VeryLongName"
```

These test cases cover scenarios where:

- The filename length exactly matches max_len.
- The function partially trims while preserving part of the extension.
- Filenames without extensions are handled correctly.
- Full trimming is applied when necessary.

You can run these tests using Hypothesis with specific settings to limit examples and increase verbosity:
$pytest - v - -settingsVerbosity.verbose, max_examples = 2test_trim_filename.py$

Comparison with LLM-Generated Test Cases: Test Case 1 - Exact Match Both your test

case and an LLM would focus on ensuring the filename remains unchanged when its length matches max_len.

Test Case 2 - Partial Extension Preservation Similar to what an LLM might generate, both approaches ensure partial extension preservation in cases of trimming.

Test Case 3 - No Extension Handling Both methods handle filenames without extensions correctly, ensuring no changes when appropriate.

Test Case 4 - Full Trimming Both strategies consider the scenario where max_len is zero and ensure no modification occurs.

Areas for Improvement:

- Confidence Assessment: Your test cases do not include testing ChatGPT's confidence levels or whether it might be overconfident (as discussed in Citation 1).
- Prompting Strategies: While both your test cases cover basic functionality, incorporating varying prompting strategies as mentioned in the paper could enhance the test suite.

Conclusion:
The four test cases are effective for verifying the core functionalities of $trim_filename$. To align more closely with the paper's methodology, additional test cases should be developed to evaluate ChatGPT's confidence levels and its response to different prompting strategies. This would ensure a more thorough assessment of how ChatGPT handles software testing scenarios as described in the citations.